

Nexus Dispatch

Requisitos das classes Java

Versão 2.0 - revisão de escopo e arquitetura

Documento atualizado conforme as decisões do projeto até 30/09/2026.

Marketplace local de produtos físicos que conecta consumidores, lojistas e entregadores. Esta revisão substitui as definições anteriores de interface por terminal e de uma compra limitada a uma única loja.

Escopo atual

Java/Maven, Spring Boot, interface HTML/CSS com templates, persistência JDBC/MySQL, pagamento simulado e confirmação transacional de compras com várias lojas. Login e autenticação permanecem fora da primeira versão.

Compra e entrega

Uma Compra reúne um ou mais Pedidos. Cada Pedido pertence a uma única Loja e gera uma OrdemEntrega própria. Produtos da mesma loja seguem juntos na entrega desse pedido; produtos de lojas diferentes não compartilham a mesma ordem.

Decisões incorporadas

Lojistas identificados por CNPJ, com validação. Entregadores com CNH obrigatória para moto ou carro; bicicleta dispensa CNH. Bairros próximos passam a integrar o escopo, com critérios ainda a refinar.

Simplificações

Todo frete tem uma taxa de intermediação calculada por um percentual global previamente configurado. Essa parcela fica com o Nexus e o restante com o entregador. O valor numérico do percentual e o cálculo do frete estão a definir. A comissão global sobre produtos permanece conforme o requisito anterior. A exigência de confirmação dupla de entrega foi retirada.

Como ler este documento

Requisito definido: decisão já acordada. Proposta técnica: organização recomendada para implementar a decisão. A definir: detalhe ainda aberto, sem implementação presumida. A existência de um arquivo Java não significa que sua lógica já esteja pronta.

1. Usuários, documentos e localização

Usuario - abstrata

Finalidade: concentrar dados comuns. Atributos: Long id, String nomeCompleto, String contato, Endereco endereco. Não pode ser instanciada diretamente. Cada usuário tem um único perfil. Consumidor, Lojista, Entregador e Administrador herdam de Usuario.

Consumidor

Finalidade: representar quem realiza compras. Relacionamento: Consumidor 1 - N Compra. Criação de compras, consultas e validações ficam nos serviços.

Lojista

Finalidade: representar o responsável por uma loja. Atributos específicos: String cnpj e Loja loja. Cada lojista tem uma loja; cada loja tem um lojista. CNPJ é obrigatório, deve ser validado antes de salvar e não pode identificar cadastros duplicados de lojistas. Guardar como texto normalizado. Validação local de formato e dígitos verificadores não comprova situação cadastral; consulta oficial externa não está definida no escopo.

Entregador

Atributos específicos: TipoVeiculo tipoVeiculo, String cnh, CategoriaCnh categoriaCnh, Bairro bairroAtuacao e boolean disponivel. Possui um tipo de veículo por vez. Bicicleta dispensa CNH; moto exige categoria A ou AB; carro exige B ou AB. Validar preenchimento, formato do documento e compatibilidade da categoria. Consulta externa de autenticidade ou situação da CNH não está definida.

Administrador

Perfil que acessa os módulos pelos mesmos serviços usados pelos demais perfis. Herda somente de Usuario. Não duplica regras de Consumidor, Lojista ou Entregador. Não representa uma implementação de autenticação nesta versão.

Endereco

Atributos: Long id, String logradouro, String numero, String complemento, Bairro bairro, String cidade. Complemento opcional. Usado por usuários e lojas. Bairro deve existir na base do Nexus. Não haverá API de CEP inicialmente.

Bairro

Atributos: Long id, String nome, String cidade, boolean atendido. Normalizar nomes nas consultas e validar contra a base própria. Bairros próximos estão no escopo: representação da proximidade, direção da relação e critérios de elegibilidade ainda serão definidos. Não presumir raio, distância ou API de mapas.

2. Comércio, compra e entrega

Loja e Produto

Loja: Long id, String nome, Endereco endereco, Lojista lojista. Produto: Long id, String nome, String descricao. Produto representa o item genérico e não guarda preço, estoque ou loja diretamente.

OfertaProduto

Atributos: Long id, Produto produto, Loja loja, BigDecimal preco, int estoque. Relaciona produto e loja. Preço e estoque não podem ser negativos; ofertas com estoque zero não aparecem como disponíveis para compra.

Compra - agrupamento do checkout

Proposta: Long id, Consumidor consumidor, LocalDateTime dataHora, List<Pedido> pedidos, MetodoPagamento metodoPagamento, StatusPagamento statusPagamento e valores agregados de produtos, frete e total. Método e status do pagamento ficam na Compra, evitando duplicação nos pedidos. Proposta: registrar o endereço de entrega usado na compra de modo que alterações futuras do cadastro não modifiquem seu histórico.

Pedido - parcela de uma loja

Atributos: Long id, Compra compra, Loja loja, LocalDateTime dataHora, List<ItemPedido> itens, BigDecimal subtotal, BigDecimal valorFrete, BigDecimal valorTotal, StatusPedido statusPedido. Uma compra gera um pedido por loja participante. O consumidor é acessível pela Compra. Todos os itens devem pertencer à loja do pedido.

ItemPedido

Atributos: Long id, Pedido pedido, OfertaProduto ofertaProduto, int quantidade, BigDecimal precoUnitario, BigDecimal subtotal. Quantidade positiva e limitada ao estoque disponível. Guardar o preço da compra; futuras alterações da oferta não alteram o histórico.

OrdemEntrega

Atributos: Long id, Pedido pedido, Entregador entregador (opcional até o aceite), BigDecimal valorFrete, BigDecimal percentualTaxaFrete, BigDecimal taxaIntermediacao, BigDecimal valorEntregador e StatusEntrega statusEntrega. Uma ordem por pedido. Todo frete aplica o mesmo percentual global vigente. Registrar o percentual aplicado e os valores calculados para preservar o histórico quando a configuração global mudar.

Aceite e conclusão

Somente entregadores disponíveis e elegíveis pela regra territorial podem aceitar. O primeiro aceite válido torna o entregador responsável; impedir dois responsáveis simultâneos. Não exigir confirmação dupla. Proposta: o entregador responsável registra a conclusão pelo fluxo de atualização de status, sem uma confirmação adicional do consumidor.

3. Enums e estados

Os valores abaixo distinguem definições já acordadas de propostas para os ciclos de vida. Os enums padronizam valores; as transições e validações pertencem aos serviços.

Enum	Valores e uso
TipoVeiculo	BICICLETA, MOTO, CARRO. Definido. Usado em Entregador.
CategoriaCnh	A, B, AB. Recorte das categorias relevantes ao projeto. Para bicicleta, a ausência de categoria é permitida.
MetodoPagamento	PIX, CARTAO, DINHEIRO. Definido. Pagamento simulado; usado em Compra.
StatusPagamento	PENDENTE, APROVADO, RECUSADO. Definido. Recusa impede a confirmação integral da compra.
StatusPedido	CONFIRMADO, EM_PREPARACAO, EM_ROTA, ENTREGUE. Valores do documento anterior, ainda propostos para validação do fluxo.
StatusEntrega	Proposta: AGUARDANDO_ENTREGADOR, ACEITA, EM_ROTA, ENTREGUE. Usado em OrdemEntrega.

StatusCompra - dispensado nesta etapa

Não criar um ciclo de vida independente sem necessidade. A situação geral pode ser consultada a partir do pagamento e dos pedidos. Se surgir uma regra própria para Compra, reavaliar a criação desse enum.

DisponibilidadeEntregador - dispensado nesta etapa

Manter boolean disponível, conforme a necessidade atual. Criar um enum apenas se o projeto precisar distinguir mais estados de disponibilidade.

Coerência entre status

Proposta: aceite da ordem não significa que o pedido já saiu da loja. Ao iniciar a rota, sincronizar Pedido e OrdemEntrega em EM_ROTA; ao concluir, sincronizar em ENTREGUE. Validar sequência e responsável pelo pedido/entrega antes de atualizar.

Pagamento e rollback

Estados PENDENTE e RECUSADO podem existir durante o processamento em memória. Não obrigam a persistir uma compra recusada: a confirmação continua sendo tudo ou nada. Não haverá fluxo de cancelamento posterior de compra confirmada nesta etapa.

4. Serviços e responsabilidades

Classe	Responsabilidade prevista
UsuarioService	Validar dados comuns, bairro atendido e cadastros; consultar usuários. Sem senha ou autenticação.
LojaService	Coordenar cadastro de lojista/loja, validar CNPJ e unicidade, garantir um lojista por loja e uma loja por lojista. Reutilizar UsuarioService para dados comuns.
EntregadorService	Validar CNH e categoria conforme o veículo; cadastrar dados específicos, definir área de atuação e disponibilidade.
ProdutoService	Consultar produtos e ofertas; controlar preço e estoque por loja; exibir apenas ofertas disponíveis.
CompraService	Orquestrar o checkout: agrupar itens por loja, validar estoque, simular pagamento, calcular totais e persistir toda a compra em uma transação.
PedidoService	Proposta: cuidar do andamento de cada pedido após sua criação e validar transições. Não executar um checkout independente nem confirmar parcialmente uma compra.
EntregaService	Criar ordens, consultar elegibilidade territorial, controlar aceite exclusivo e atualizar andamento/conclusão.
FinanceiroService	Calcular comissão global sobre produtos, frete, taxa global sobre cada frete, repasse ao entregador e totais. Usar BigDecimal.

Limites entre camadas

Entidades representam dados e relações. Controllers recebem solicitações das telas e chamam serviços. Services aplicam regras e coordenam DAOs. DAOs executam persistência e consultas. Evitar duplicar validações e operações de negócio em controllers ou entidades.

Definições financeiras pendentes

A taxa do frete é global, não varia por lojista ou entregador, e deve estar configurada antes de confirmar compras. Fórmulas: $\text{taxaIntermediacao} = \text{valorFrete} \times \text{percentualGlobal} / 100$; $\text{valorEntregador} = \text{valorFrete} - \text{taxaIntermediacao}$. A taxa é uma parcela do frete, não uma cobrança adicional sobre ele. O percentual numérico, sua faixa permitida, arredondamento e regra de cálculo do frete ainda precisam ser definidos. Manter separada a comissão global sobre produtos, cujo percentual, base e responsável pelo pagamento também precisam ser definidos.

5. Persistência e transação

Manter JDBC/MySQL conforme o documento original, integrado à aplicação Spring Boot. Usar a convenção de nomes Dao de forma consistente.

Classe	Responsabilidade prevista
UsuarioDao	Dados comuns e dados específicos dos perfis; a estratégia de tabelas para herança será definida na modelagem.
BairroDao	Consulta da base de bairros atendidos e, após refinamento, das relações de proximidade.
LojaDao / ProdutoDao	Persistência de lojas e produtos genéricos.
OfertaProdutoDao	Ofertas, preços, estoque e atualização segura contra compras concorrentes.
CompraDao	Compra, consumidor, dados do pagamento simulado e totais agregados.
PedidoDao / ItemPedidoDao	Pedidos por loja, itens e preços históricos.
OrdemEntregaDao	Ordens, vínculo com entregador, aceite exclusivo e status.

Conexões e infraestrutura

Proposta: usar o DataSource e o gerenciamento de transações do Spring Boot. DatabaseConfig só será necessária se houver configuração Java específica; não criar ConexaoFactory em paralelo sem necessidade. DAOs devem participar da mesma transação e não confirmar operações isoladamente. Versões e dependências serão definidas no pom.xml.

Confirmação atômica da compra

1. Validar consumidor, endereço e itens selecionados.
2. Agrupar os itens por loja e validar preços/estoque.
3. Calcular valores e simular o pagamento.
4. Criar Compra, Pedidos, ItemPedidos e OrdensEntrega.
5. Atualizar estoque e registrar o método/status do pagamento.
6. Confirmar tudo com COMMIT apenas se todas as etapas forem concluídas.

Falha em qualquer etapa

Executar ROLLBACK integral: nenhuma compra, pedido, item ou ordem dessa operação permanece gravada e nenhuma baixa de estoque permanece aplicada. Validar e atualizar estoque de modo seguro contra concorrência; uma consulta isolada antes da baixa não é suficiente.

A atomicidade vale para a confirmação do checkout. Não significa esperar a entrega física de todas as lojas para confirmar a transação. Falhas posteriores de entrega não são tratadas como rollback dessa transação já concluída.

6. Interface web e relacionamentos

Interface

Spring Boot com HTML/CSS; proposta de templates com Thymeleaf. Substitui menus de terminal. Sem autenticação na primeira versão; a forma de selecionar o usuário de demonstração ainda deve ser definida. O perfil administrador acessa os mesmos serviços.

Controller	Papel
HomeController	Página inicial e acesso aos módulos.
ConsumidorController	Catálogo, carrinho, checkout e consulta das compras.
LojistaController	Loja, ofertas e andamento dos pedidos.
EntregadorController	Disponibilidade, ordens elegíveis, aceite e atualização da entrega.
AdministradorController	Acesso administrativo aos módulos por meio dos serviços.

Cardinalidades do domínio

Usuario é a superclasse dos quatro perfis.

Consumidor 1 - N Compra.

Compra 1 - N Pedido; cada pedido pertence a uma única compra e loja.

Lojista 1 - 1 Loja.

Loja 1 - N OfertaProduto; Produto 1 - N OfertaProduto.

Pedido 1 - N ItemPedido; cada item referencia uma OfertaProduto.

Pedido 1 - 1 OrdemEntrega.

Entregador 1 - N OrdemEntrega ao longo do tempo; cada ordem tem zero ou um responsável.

Endereco referencia Bairro; Usuario e Loja referenciam Endereco.

Representação em Java e no banco

Usar referências entre objetos e coleções quando necessárias. Usar chaves estrangeiras no banco. As cardinalidades de Compra e Pedido descrevem registros confirmados; durante a montagem, objetos podem estar incompletos. Não carregar listas em ambos os lados de toda associação sem necessidade.

Histórico e integridade

Preço do ItemPedido deve permanecer histórico. Proposta: preservar também o endereço usado na entrega. O pedido só pode conter ofertas de sua loja. A atribuição do entregador deve impedir aceite concorrente por duas pessoas.

7. Estrutura Java e próximos ajustes

Pacote	Classes previstas
model	Usuario, Consumidor, Lojista, Entregador, Administrador, Endereco, Bairro, Loja, Produto, OfertaProduto, Compra, Pedido, ItemPedido, OrdemEntrega.
enums	TipoVeiculo, CategoriaCnh, MetodoPagamento, StatusPagamento, StatusPedido, StatusEntrega.
service	UsuarioService, LojaService, EntregadorService, ProdutoService, CompraService, PedidoService, EntregaService, FinanceiroService.
dao	UsuarioDao, BairroDao, LojaDao, ProdutoDao, OfertaProdutoDao, CompraDao, PedidoDao, ItemPedidoDao, OrdemEntregaDao.
controller	HomeController, ConsumidorController, LojistaController, EntregadorController, AdministradorController.
database	DatabaseConfig apenas se houver necessidade de configuração específica.
Pacote principal	NexusDispatchApplication: inicialização do Spring Boot.

Ajustes necessários na pasta atual

Adicionar Endereco, Bairro, StatusPagamento, UsuarioService, FinanceiroService, BairroDao e PedidoService. Dispensar StatusCompra e DisponibilidadeEntregador nesta etapa. Avaliar DatabaseConfig na configuração do banco. Essas mudanças estão documentadas aqui; os arquivos do projeto não foram alterados nesta revisão.

Recursos e testes

Estrutura prevista: src/main/resources/templates e static/css; configuração da aplicação e scripts de banco conforme a implementação. Testes devem cobrir rollback entre lojas, estoque concorrente, preço histórico, CNPJ, CNH por veículo, aceite exclusivo e elegibilidade territorial. Os arquivos atuais de teste ainda são estruturas vazias.

Pendências para implementação

Refinar bairros próximos; validar estados e transições de pedido/entrega; definir cálculo do frete, percentual global da taxa sobre frete e comissão sobre produtos; detalhar validação dos documentos e persistência dos perfis; definir versões e configuração do ambiente. Não introduzir consulta oficial externa de documentos sem decisão específica.

Próxima etapa sugerida: validar os valores dos enums e implementar suas regras básicas antes de preencher as entidades e os serviços.